Week- 2

Backend Engineering Launchpad

Pawan Panjwani

Airtribe

Backend Development Design patterns and low level design

Airtribe

Agenda

- What is low level design ?
- Key considerations in low level design
- UML diagrams
- Use case diagrams
- Class diagrams
- Low Level Design - Parking Lot

Airtribe

What is low level design

- **What is Low Level Design?**
- **Definition**: LLD involves detailing the system's components, their relationships, and interactions. It is the blueprint for developers to write code.
- **Focus**: Classes, methods, data structures, algorithms, and their interactions.
- **Objective**: Translate high-level architecture into detailed design for implementation.

Key Considerations of LLD

Requirements

- **Functional Requirements**: What the system should do.
- **Non-Functional Requirements**: Performance, security, scalability.

Design Principles

Design Principles

- **SOLID Principles**: Ensure good object-oriented design.
- **DRY, KISS, YAGNI**: Keep the design simple and efficient.
- **Design Patterns**: Reusable solutions to common problems.

Key Design Principles

- Solid Principles
- DRY (Don’t repeat yourself)
- KISS (Keep it simple, stupid)
- YAGNI (You aren’t gonna need it)
- Composition over Inheritance
- Encapsulate what changes

Advanced Design Principles

- Law of Demeter
- Encapsulation
- Polymorphism
- Abstraction
- Modularity
- Separation of concerns
- Cohesion and coupling

Design patterns – Categories

- Structural design patterns - Deal with object composition.
- Creational design patterns - Deal with object creation mechanisms.
- Behavioural design patterns - Deal with object interaction and responsibility.

Design patterns – Categories

- Structural design patterns - Deal with object composition.
- Creational design patterns - Deal with object creation mechanisms.
- Behavioural design patterns - Deal with object interaction and responsibility.

Creational Design Patterns

- Singleton **
- Factory **
- Abstract factory **
- Builder **
- Prototype

Structural Design Patterns

- Adapter **
- Bridge **
- Composite **
- Decorator **
- Facade
- Flyweight
- Proxy

Behavioral Design Patterns

- Chain of responsibility **
- Command **
- Interpreter
- Iterator **
- Mediator
- Momento
- Observer **
- State **
- Strategy **

Airtribe

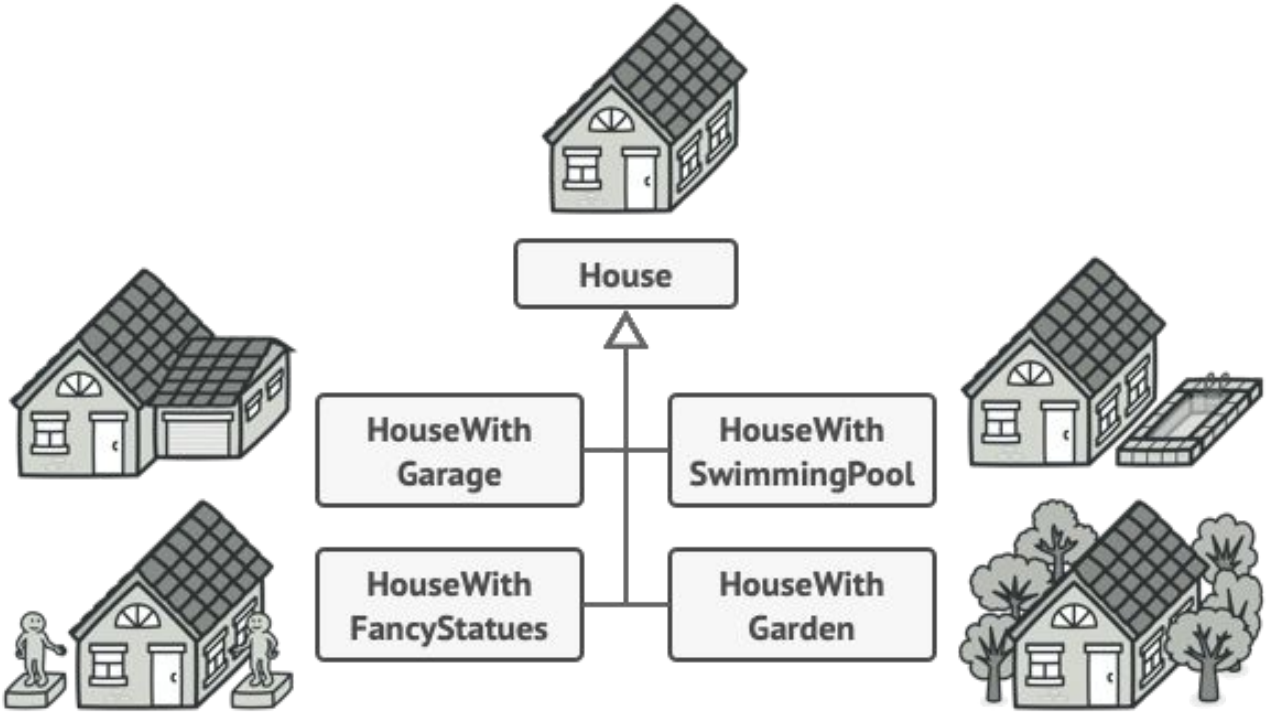
CDP – Builder pattern

The **Builder Pattern** is a creational design pattern used to construct complex objects step by step. It allows you to create an object with different representations (configurations) without having to specify the exact class of the object that is being created.

Intent:

- The builder pattern helps separate the construction of a complex object from its representation. This allows the same construction process to create different representations of an object.
- It is particularly useful when objects have many optional components or configurations and we want to avoid telescoping constructor anti-pattern (i.e., constructors with too many parameters).

CDP – Builder pattern



Airtribe

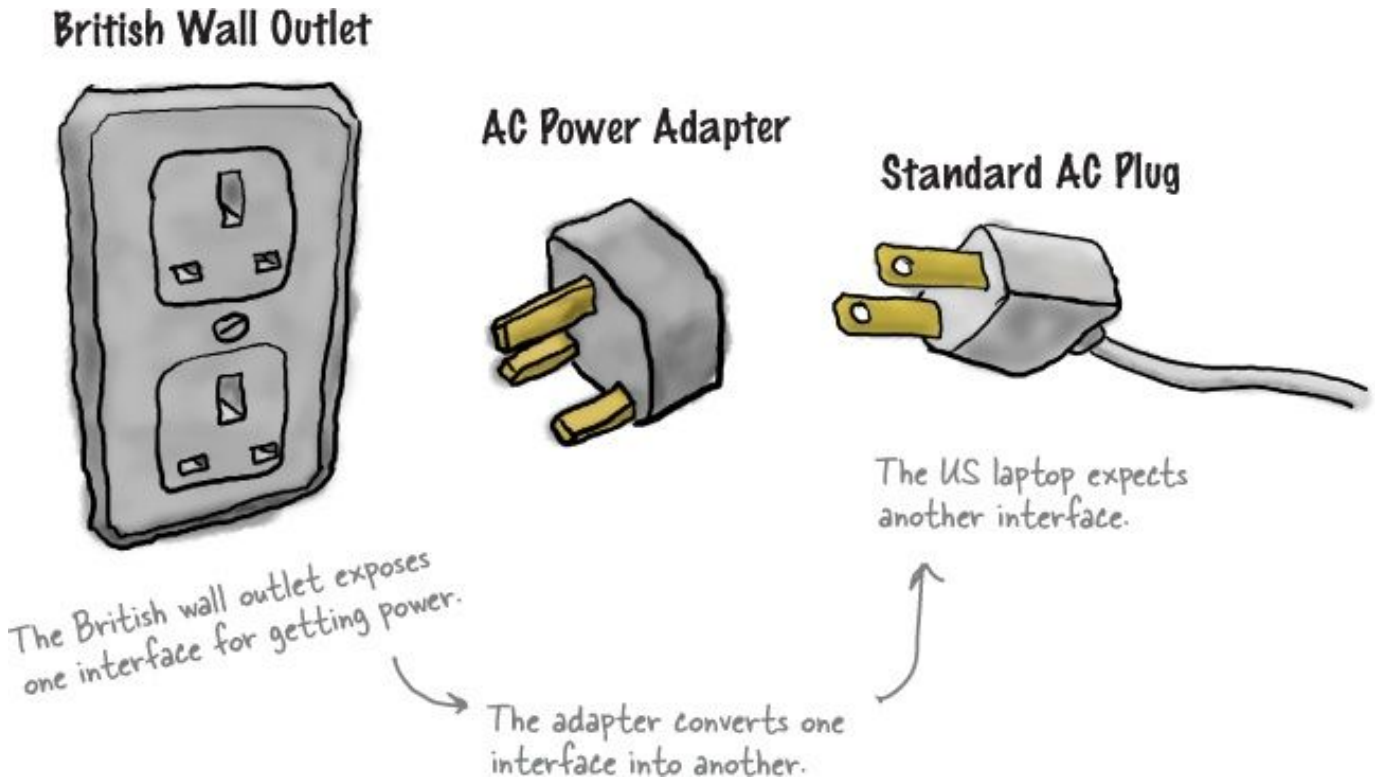
SDP – Adapter pattern

Definition: The **Adapter Pattern** is a structural design pattern that allows incompatible interfaces to work together. It acts as a bridge between two interfaces, enabling them to communicate with each other. The adapter converts one interface into another, making the existing system compatible with new interfaces without modifying the original code.

Intent:

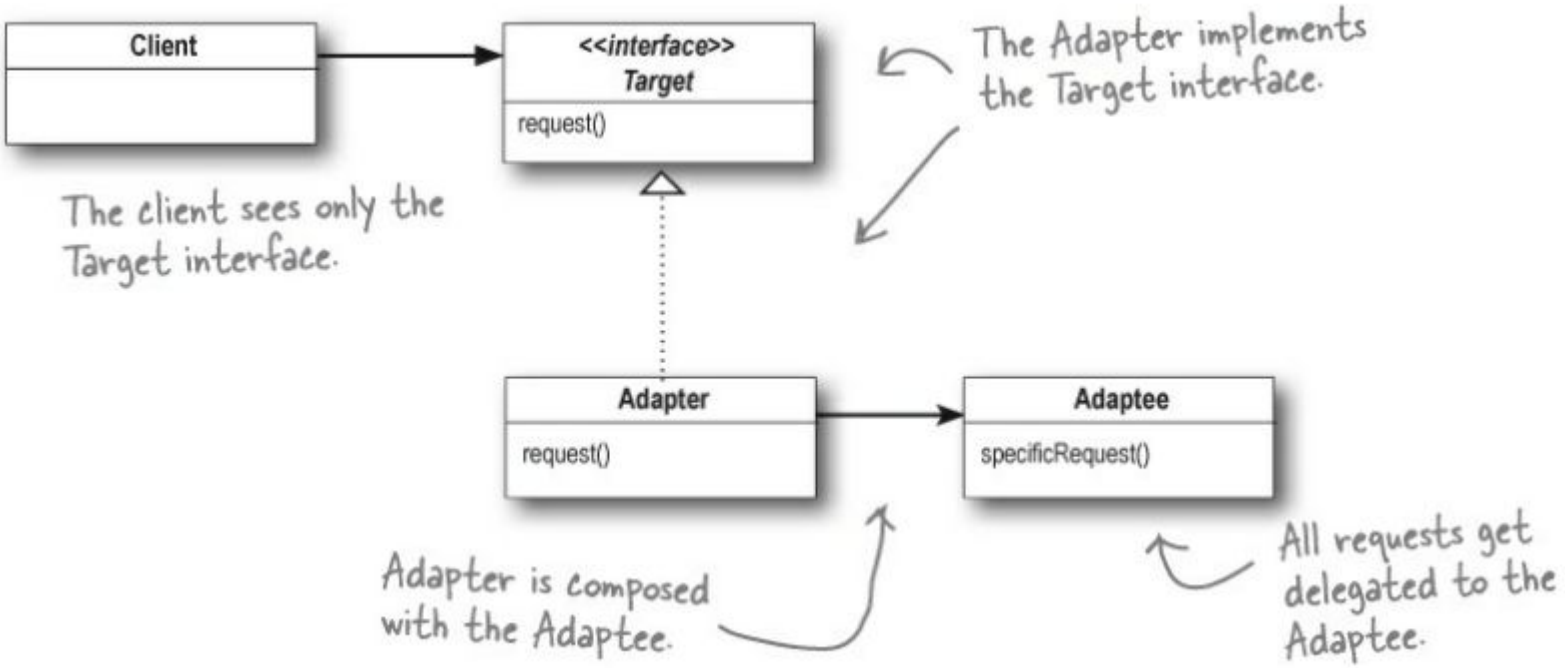
- The adapter pattern allows a class to work with an interface it wasn't originally designed to work with by adapting one interface to another.
- It is used when you need to integrate a new system into an existing one but don't want to modify the original code.

SDP – Adapter pattern



Airtribe

SDP – Adapter pattern



Airtribe

SDP – Adapter pattern

- Adapter pattern - Allow incompatible interfaces to work together.

```java
public interface Target {
    void request();
}

public class Adaptee {
    public void specificRequest() {
        System.out.println("Specific request");
    }
}

public class Adapter implements Target {
    private Adaptee adaptee;

    public Adapter(Adaptee adaptee) {
        this.adaptee = adaptee;
    }

    public void request() {
        adaptee.specificRequest();
    }
}

public class Client {
    public static void main(String[] args) {
        Adaptee adaptee = new Adaptee();
        Target target = new Adapter(adaptee);
        target.request();
    }
}
```

Airtribe

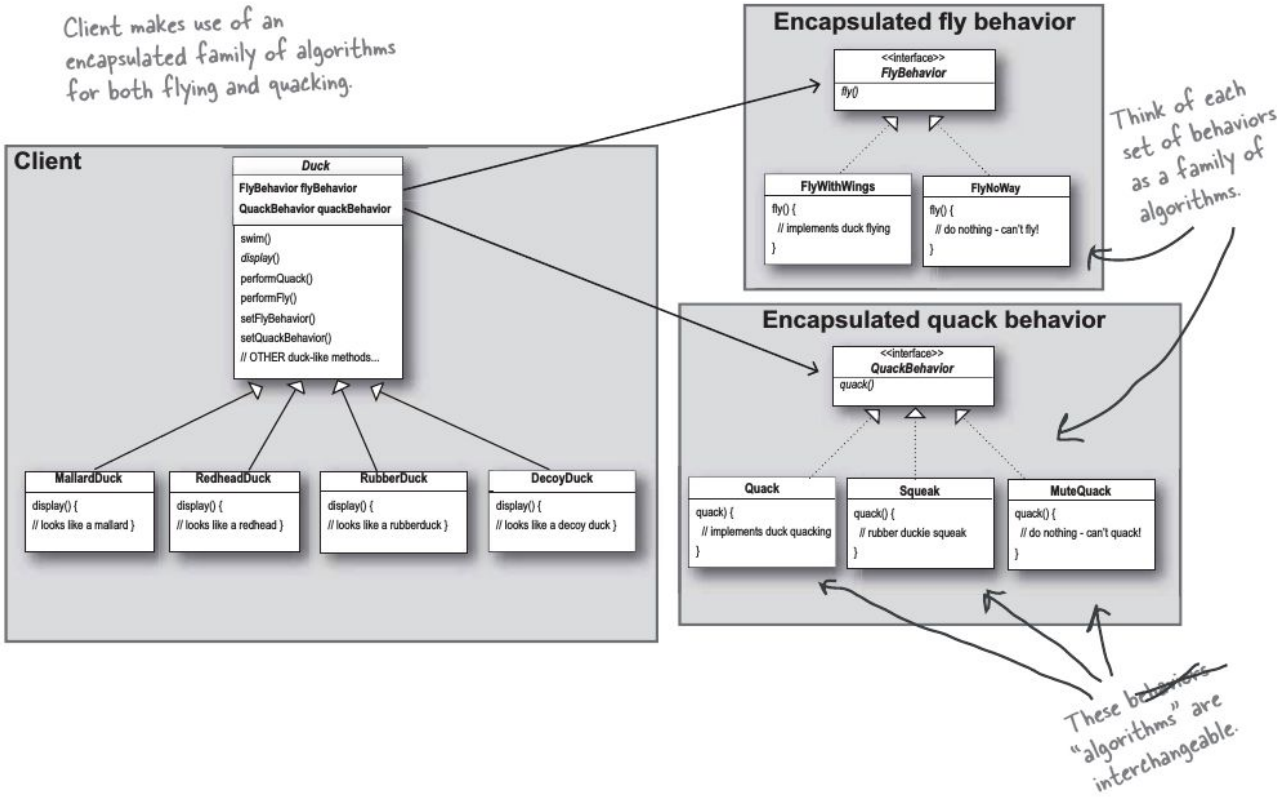
BDP – Strategy pattern

- The **Strategy Pattern** is a behavioral design pattern that defines a family of algorithms and makes them interchangeable. This pattern allows a client to choose which algorithm to use at runtime. The strategy pattern is useful when a class has multiple ways of performing an operation, and you want to avoid using multiple conditional statements (like if-else or switch).

  Intent:

  - It defines a set of algorithms, encapsulates each one, and makes them interchangeable.
  - It allows a class to delegate the responsibility of selecting an algorithm to a strategy object.

BDP – Strategy pattern



Airtribe

BDP – Strategy pattern

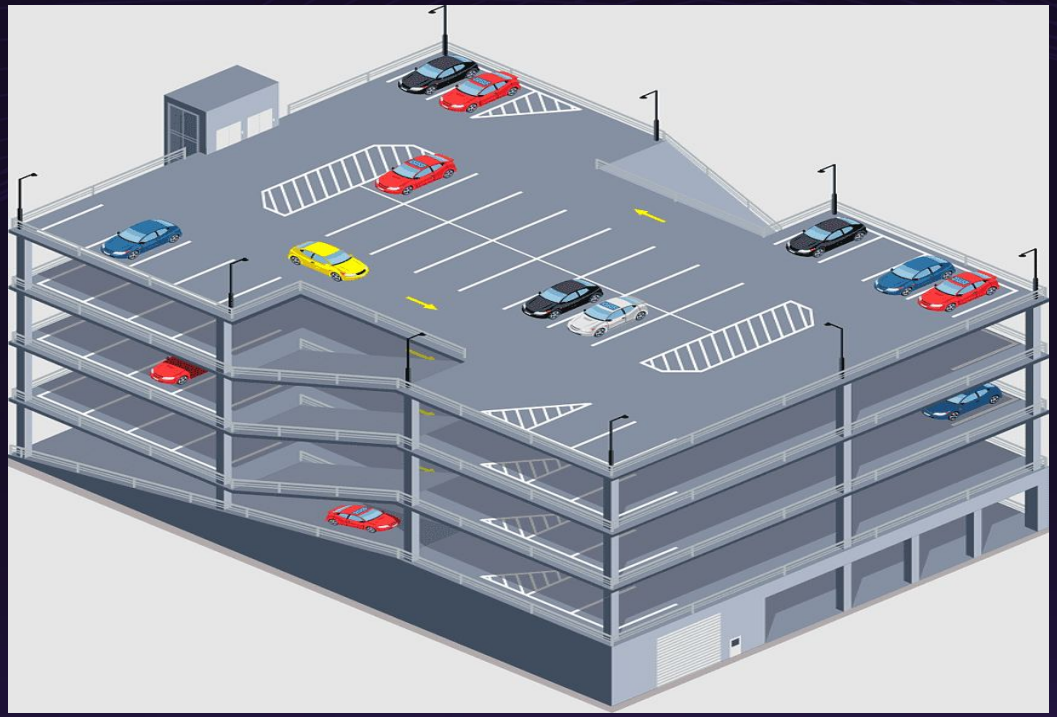
```java
public interface Strategy {
    void execute();
}

public class ConcreteStrategyA implements Strategy {
    public void execute() {
        System.out.println("Strategy A execution");
    }
}

public class ConcreteStrategyB implements Strategy {
    public void execute() {
        System.out.println("Strategy B execution");
    }
}

public class Context {
    private Strategy strategy;

    public void setStrategy(Strategy strategy) {
        this.strategy = strategy;
    }

    public void executeStrategy() {
        strategy.execute();
    }
}

public class Client {
    public static void main(String[] args) {
        Context context = new Context();

        Strategy strategyA = new ConcreteStrategyA();
        context.setStrategy(strategyA);
        context.executeStrategy();

        Strategy strategyB = new ConcreteStrategyB();
        context.setStrategy(strategyB);
        context.executeStrategy();
    }
}
```

Airtribe

Parking Lot Design

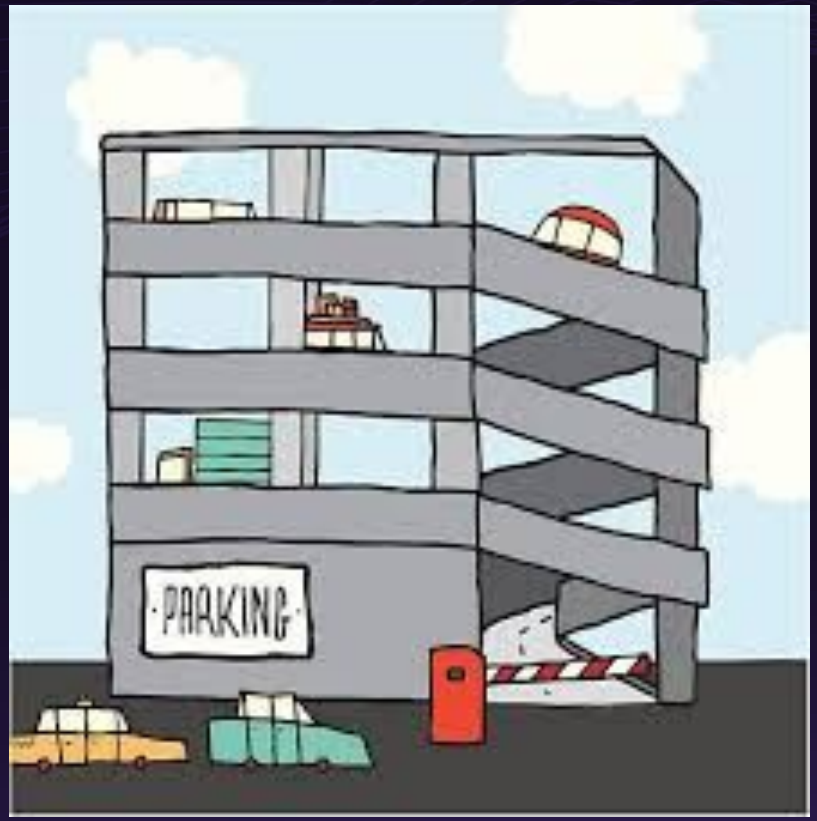
Airtribe

Imagine how it looks like



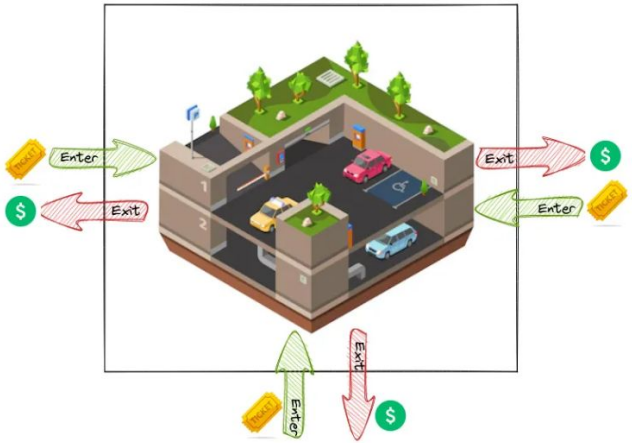
Airtribe

Imagine how it looks like



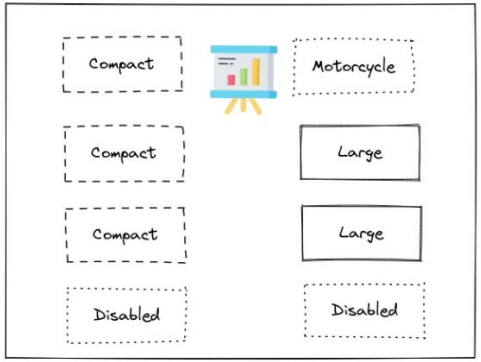
Airtribe

Parking Lot



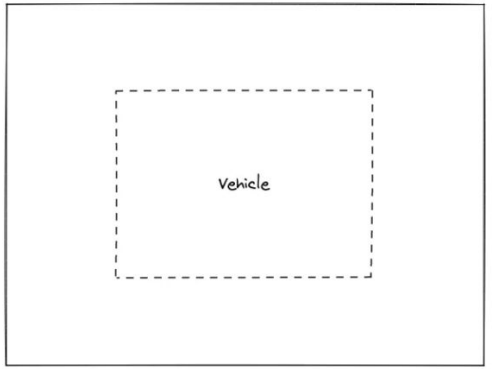
1. Parking floors
2. Entry panel
3. Exit panel

Parking floor



1. Parking spots
2. Display board - Floor

Parking spot



1. Vehicle

Functional Requirements – Parking Lot

- Ability to park and unpark the vehicles
- Parking should be allocated closer to the entry and exit terminals
- Supports different types of vehicles (Car, Bike, Truck, 2 wheeler)
- Calculates parking fees based on the duration
- Keeps track of available parking spots
- Should not allow more vehicles than the capacity of the parking lot
- Separate parking spots for - Handicapped parking, Compact cars, Large vehicles, motorcycles
- Parking fees can be paid using credit cards and cash

Important requirements

- Parking lot system should be extensible and can be used for installation of different parking lot systems.
- Code modification should be minimal for the extension of the parking lot for different categories

Actors in the system

- Parking Lot System
- Entry and Exit terminals
    - Printers
    - Payment processors
- Parking Spot
- Ticket
- Database
- Monitoring System
- Vehicle

Got Questions?

Airtribe

Thank you

Airtribe